

Betrayal Game — Wave 2: Identity & Stats Layer

Replit Agent Engineering Prompts

Build order matters. Run these sequentially — each prompt depends on the previous one being complete.

Prerequisites: Wave 1 Prompts 1 (WS Router) and 2 (SQLite) must be complete before starting Wave 2.

PROMPT 1: Player Profiles & Persistent Identity

You are working on a multiplayer social deduction game called **Betrayal Game**. The tech stack is Node.js + TypeScript on the backend, React 19 + TypeScript + Vite on the frontend, CSS Modules for styling, and `better-sqlite3` for persistence (added in a prior task). The server runs on port 5000 and communicates via WebSockets.

Context

Currently, players type their name each session and have no persistent identity. There are no accounts. The goal is to introduce lightweight persistent player profiles — no passwords, no email, no OAuth. A player is identified by a **device fingerprint token** stored in `localStorage`, linked to a chosen display name. This is frictionless for a fixed friend group of 10–20 people who trust each other's devices.

What needs to be done

1. Client-side: Device fingerprint

In `client/src/utils/identity.ts` (create this file), implement:

```
export function getOrCreateDeviceToken(): string
// Reads from localStorage key 'betrayal_device_token'
// If not present, generates a UUID using crypto.randomUUID() and stores it
// Returns the token

export function getSavedPlayerName(): string | null
// Reads from localStorage key 'betrayal_player_name'
```

```
// Returns null if not set

export function savePlayerName(name: string): void
// Saves to localStorage key 'betrayal_player_name'

export function getPlayerProfile(): { deviceToken: string; playerName: string | n
// Returns both values together
```

2. Server-side: Player profile DB table

Add to `src/db/store.ts` (extending the existing SQLite store from Wave 1):

```
CREATE TABLE IF NOT EXISTS player_profiles (
  device_token TEXT PRIMARY KEY,
  player_name TEXT NOT NULL,
  created_at INTEGER NOT NULL,
  last_seen_at INTEGER NOT NULL
);
```

Export these functions:

```
export function upsertPlayerProfile(deviceToken: string, playerName: string): voi
// INSERT OR REPLACE – updates last_seen_at on every call

export function getPlayerProfile(deviceToken: string): PlayerProfile | null
// Returns { deviceToken, playerName, createdAt, lastSeenAt } or null

export function getPlayerProfileByName(playerName: string): PlayerProfile | null
// Used for name conflict detection
```

3. New WebSocket events

Add to `src/game/types.ts`:

```
// Client → Server
{ type: 'C2S_IDENTIFY', payload: { deviceToken: string; playerName: string } }

// Server → Client
{ type: 'S2C_IDENTITY_CONFIRMED', payload: { deviceToken: string; playerName: str
{ type: 'S2C_IDENTITY_ERROR', payload: { message: string } }
```

Handle `C2S_IDENTIFY` in the WS router **before** `C2S_CREATE_GAME` or `C2S_JOIN_GAME`. When received:

- **Validate:** `playerName` must be 2–20 characters, alphanumeric + spaces only
- Call `upsertPlayerProfile(deviceToken, playerName)`
- Respond with `S2C_IDENTITY_CONFIRMED`, setting `isReturningPlayer: true` if the profile already existed
- Store `deviceToken` on the WebSocket connection context alongside `currentPlayerId` — it will be written to game history records later

4. Client-side: Identity flow in the UI

Modify `client/src/components/Lobby.tsx`:

- On mount, call `getPlayerProfile()` from `identity.ts`
- If `playerName` is already saved in `localStorage`, pre-fill the name input field with it
- After the user types/confirms their name, send `C2S_IDENTIFY` with their `deviceToken` and `playerName` before sending `C2S_CREATE_GAME` or `C2S_JOIN_GAME`
- Wait for `S2C_IDENTITY_CONFIRMED` before proceeding to create or join
- If `isReturningPlayer: true`, show a subtle “Welcome back, [name]” message
- Call `savePlayerName(name)` after confirmed so it persists for next time

5. Integration with game join flow

When a player joins or creates a game, attach their `deviceToken` to the `Player` object in game state. Add `deviceToken?: string` to the `Player` type in `src/game/types.ts`. This field is used by the stats engine (Prompt 2) to link game outcomes to persistent profiles. It must never be broadcast to other players — strip it from any `S2C_PLAYER_JOINED` or `S2C_GAME_JOINED` payloads.

Constraints

- No authentication system, passwords, or email — device token only
- `deviceToken` must never appear in any broadcast payload visible to other players
- `localStorage` keys must use the `betrayal_` prefix to avoid collisions
- TypeScript strict mode must remain satisfied
- Do not change any existing game logic in `src/game/manager.ts`

Definition of done

- Returning player sees their name pre-filled on the Lobby screen
 - `C2S_IDENTIFY` is handled before any game action
 - `player_profiles` table exists and is populated after a game session
 - `deviceToken` is stored on `Player` objects in game state but never broadcast
 - TypeScript compiles cleanly
-

PROMPT 2: Game History & Stats Engine

You are working on **Betrayal Game** (Node.js + TypeScript backend, React 19 + Vite frontend, `better-sqlite3` SQLite persistence). Player profiles with device tokens are now persisted (added in Wave 2 Prompt 1).

Context

Every completed game should write a structured record to the database. This is the foundation for player profiles, leaderboards, and post-game summaries. The game emits `S2C_GAME_END` with a `winner` field (`'TRAITORS'` or `'FAITHFUL'`) and a `history` array of round events. The `GameState` type has `players` (with `role`, `isAlive`, `deviceToken`), `currentRound`, and `winner`.

What needs to be done

1. Database schema

Add these tables to `src/db/store.ts`:

```
CREATE TABLE IF NOT EXISTS game_records (  
  id                TEXT PRIMARY KEY,      -- UUID  
  session_id       TEXT NOT NULL,  
  started_at       INTEGER NOT NULL,      -- Unix ms  
  ended_at         INTEGER NOT NULL,      -- Unix ms  
  winner           TEXT NOT NULL,         -- 'TRAITORS' | 'FAITHFUL'  
  total_rounds     INTEGER NOT NULL,  
  player_count     INTEGER NOT NULL,  
  traitor_count    INTEGER NOT NULL,  
  history_json     TEXT NOT NULL          -- serialised GameState.history array  
);
```

```

CREATE TABLE IF NOT EXISTS player_game_records (
  id            TEXT PRIMARY KEY,          -- UUID
  game_id       TEXT NOT NULL REFERENCES game_records(id),
  device_token  TEXT NOT NULL,
  player_name   TEXT NOT NULL,            -- snapshot at time of game
  role          TEXT NOT NULL,            -- 'TRAITOR' | 'FAITHFUL'
  outcome       TEXT NOT NULL,            -- 'WON' | 'LOST'
  survived      INTEGER NOT NULL,         -- 1 if alive at game end, 0 if not
  was_banished  INTEGER NOT NULL,         -- 1 if banished by vote
  was_murdered  INTEGER NOT NULL,         -- 1 if murdered at night
  votes_cast    INTEGER NOT NULL,         -- total votes this player submitted
  votes_received INTEGER NOT NULL,        -- total votes this player received
  rounds_played INTEGER NOT NULL
);

```

2. Stats query functions

Add to `src/db/store.ts`:

```

export function saveGameRecord(record: GameRecord): void
// Inserts into game_records and all player_game_records in a transaction

export function getPlayerStats(deviceToken: string): PlayerStats | null
// Returns aggregated stats for one player:
// { gamesPlayed, winsAsTraitor, lossesAsTraitor, winsAsFaithful, lossesAsFaithful,
//   totalSurvived, totalBanished, totalMurdered, totalVotesCast, totalVotesReceived,
//   winRate: number (0-1), traitorWinRate: number, faithfulWinRate: number,
//   averageRoundsPlayed: number }

export function getRecentGames(deviceToken: string, limit: number): GameSummary[]
// Returns last N games for this player, most recent first
// GameSummary: { gameId, sessionId, endedAt, winner, role, outcome, playerCount,
//               ... }

export function getLeaderboard(metric: 'winRate' | 'gamesPlayed' | 'traitorWins')
// Returns top 20 players sorted by the given metric
// LeaderboardEntry: { deviceToken, playerName, value: number, gamesPlayed: number }
// Only include players with >= 3 games played to filter noise

export function getGlobalStats(): GlobalStats
// { totalGamesPlayed, totalPlayersEver, faithfulWinRate, traitorWinRate, average

```

3. Trigger on game end

In `src/ws/router.ts`, in the `C2S_CONTINUE_TO_DAY` handler where `S2C_GAME_END` is broadcast, after the broadcast call:

```
writeGameRecord(finalGameState, sessionId)
```

Create `src/game/recordWriter.ts` that exports `writeGameRecord(gameState: GameState, sessionId: string): void`. This function:

- Derives `started_at` from `gameState.history[0].timestamp` (or `Date.now() - gameState.currentRound * 300000` as a fallback)
- Sets `ended_at` to `Date.now()`
- Determines each player's outcome based on their role vs `gameState.winner`
- Derives `was_banished` and `was_murdered` from `gameState.history` events
- Derives `votes_cast` and `votes_received` by scanning all `VOTE` events in `gameState.history`
- Skips players with no `deviceToken` (guests who didn't identify)
- Wraps everything in a single SQLite transaction via `saveGameRecord`

4. New WebSocket events

Add to `src/game/types.ts`:

```
// Client → Server
{ type: 'C2S_GET_PLAYER_STATS', payload: { deviceToken: string } }
{ type: 'C2S_GET_LEADERBOARD', payload: { metric: 'winRate' | 'gamesPlayed' | 'tr

// Server → Client
{ type: 'S2C_PLAYER_STATS', payload: PlayerStats & { recentGames: GameSummary[] } }
{ type: 'S2C_LEADERBOARD', payload: { metric: string; entries: LeaderboardEntry[] }
```

These can be requested from the Lobby screen (pre-game) or from the Game End screen. The server should respond only to the requesting WebSocket, not broadcast.

Constraints

- All DB writes at game end must be in a single transaction — partial writes are not acceptable
- If a game ends without any identified players (all guests), still write the `game_records` row but write zero `player_game_records` rows
- Do not modify `src/game/manager.ts`

- TypeScript strict mode must be satisfied — define all types in `src/game/types.ts` or `src/db/types.ts`

Definition of done

- `game_records` and `player_game_records` are populated after every completed game
 - `getPlayerStats` returns accurate aggregated data
 - `C2S_GET_PLAYER_STATS` and `C2S_GET_LEADERBOARD` are handled in the WS router
 - TypeScript compiles cleanly
-

PROMPT 3: Post-Game Summary Screen

You are working on **Betrayal Game** (Node.js + TypeScript backend, React 19 + Vite frontend, CSS Modules). The game history engine and player stats are now persisted (Wave 2 Prompts 1 & 2 complete).

Context

When a game ends, `S2C_GAME_END` is broadcast with: `winner ( 'TRAITORS' | 'FAITHFUL' )`, `history` (array of round events), and the current `players` array. Currently, `client/src/components/GameEnd.tsx` shows a basic win/loss screen. This prompt replaces it with a cinematic post-game summary that makes every game feel like it meant something — the moment people screenshot and share.

What needs to be done

1. Replace `GameEnd.tsx` with a multi-stage animated summary

The screen plays through five stages automatically, with a “Continue” button available after stage 3 for players who want to skip ahead. Stages advance on a timer or user tap.

Stage 1 — Winner announcement (4 seconds) Full-screen dramatic reveal. If Traitors won: dark crimson background, the word “BETRAYED” in large gothic type, then “The Traitors win.” If Faithful won: gold background, “FAITHFUL PREVAIL”, then “The Traitors have been defeated.” Animate in with a slow fade + scale from 0.9 to 1.0.

Stage 2 — Role reveal (6 seconds) “The truth is revealed.” Header. Then each player card slides in one by one (staggered 300ms delay each), flipping to show their true role. Traitors get a dark card with a dagger icon. Faithful get a lighter card. Players who were banished or

murdered show as faded/greyed out. This is the first time Faithful players see who the Traitors were.

Stage 3 — The story: round-by-round recap (auto-advances, 3 seconds per round) A scrolling timeline showing each round:

- Round number and phase
- Who was banished (voted out) and what role they were
- Who was murdered at night and what role they were
- If a challenge occurred: who won and whether a shield was awarded A subtle progress bar shows position in the timeline. After the last round, pause 2 seconds then advance.

Stage 4 — Game MVP cards (5 seconds) Show 2–3 stat highlights from this specific game (not all-time stats):

- “Most Votes Received” — player name + count (with note if they survived anyway)
- “Most Suspicious” — player who received votes in the most rounds
- “Silent Survivor” — player who received zero votes and survived (if anyone qualifies)
- “The Architect” — the Traitor who cast the most murder votes (only shown if Traitors won) Each card animates in with a subtle slide-up.

Stage 5 — Play Again / Lobby Two buttons: “Play Again” (sends `C2S_CREATE_GAME` with same host) and “Return to Lobby” (resets local state). Show a small “Your stats have been saved” confirmation if the player has an identity (deviceToken present). Below the buttons, show a teaser: “View your full stats from the lobby.”

2. Derive MVP stats client-side

The `S2C_GAME_END` payload includes `history` and `players`. Compute MVP stats in a utility function `client/src/utils/gameStats.ts`:

```
export function computeGameMVPs(players: Player[], history: HistoryEvent[]): GameMVPs {

  type GameMVPs = {
    mostVotesReceived: { player: Player; count: number } | null
    mostSuspicious: { player: Player; roundsTargeted: number } | null
    silentSurvivor: Player | null
    theArchitect: { player: Player; murderVotesCast: number } | null // Traitor on
  }
```


Do not call the server for these — compute from the data already in the `S2C_GAME_END` payload.

3. Styling requirements

Use CSS Modules (`GameEnd.module.css`). Key design tokens to use:

- Stage 1 Traitor win: background `#1a0a0a` , accent `#8b1a1a` , text `#f0e6d3`
- Stage 1 Faithful win: background `#0a0f1a` , accent `#c9a84c` , text `#f0f4ff`
- Role cards: Traitor `#2d0a0a` with `#8b1a1a` border; Faithful `#0a1a2d` with `#1a4a8b` border
- Timeline: dark background `#111` , round markers in `#c9a84c`
- All animations use `@keyframes` — no JS animation libraries
- Must look good on mobile (375px width minimum) — stages are full-screen, portrait-optimised

4. Timing controls

Expose these as constants at the top of `GameEnd.tsx` so they can be easily tuned:

```
const STAGE_1_DURATION_MS = 4000
const STAGE_2_CARD_STAGGER_MS = 300
const STAGE_3_ROUND_DURATION_MS = 3000
const STAGE_4_DURATION_MS = 5000
```

Constraints

- All five stages must be implemented — do not stub or skip any
- Do not use any external animation libraries (no Framer Motion, no GSAP) — CSS animations only
- Do not make any new WebSocket calls from this component — all data comes from the `S2C_GAME_END` payload already in state
- TypeScript strict mode must be satisfied
- Must handle edge cases: games with 0 murders (Faithful banished all Traitors), games with 0 banishments (shouldn't happen but defend against it)

Definition of done

- All five stages render and auto-advance correctly
 - Role reveal correctly shows Traitor/Faithful for every player
 - MVP cards compute correctly from game history
 - Looks polished on both desktop and mobile (375px+)
 - TypeScript compiles cleanly, no console errors
-

PROMPT 4: Player Profile & Hall of Fame

You are working on **Betrayal Game** (Node.js + TypeScript backend, React 19 + Vite frontend, CSS Modules, `better-sqlite3` SQLite). Player stats are now persisted and queryable (Wave 2 Prompts 1–2 complete).

Context

The Lobby currently shows a name input and a room code. This prompt adds a Profile drawer and a Hall of Fame screen accessible from the Lobby, giving the fixed friend group visibility into each other's all-time records. This is the feature that creates ongoing competition and makes people want to play another round.

What needs to be done

1. Profile drawer in the Lobby

Add a profile icon button (top-right of Lobby screen) that opens a slide-up drawer (CSS-only, no library). The drawer shows the current player's stats, fetched via

`C2S_GET_PLAYER_STATS` on open.

The drawer has two tabs:

"My Stats" tab: Displayed as a stat grid (2 columns):

- Games Played
- Overall Win Rate (as percentage)
- Wins as Traitor / Games as Traitor
- Wins as Faithful / Games as Faithful
- Times Banished

- Times Murdered
- Times Survived

Below the grid, a “Recent Games” list showing the last 5 games:

- Role icon (dagger or shield)
- Outcome (Won / Lost) in colour
- Date (relative: “2 days ago”, “last week”)
- Player count and round count

If fewer than 3 games played, show an encouraging message: “Play a few more games to unlock your full stats.”

“Achievements” tab (aspirational — implement the display, seed with 6 achievements): Show achievement cards with icon, name, description, and locked/unlocked state. Locked achievements show the name but hide the description with “???”. Seed with these six:

ID	Name	Description	Unlock condition
first_blood	First Blood	Complete your first game	gamesPlayed >= 1
master_traitor	Master Traitor	Win 3 games as a Traitor	winsAsTraitor >= 3
faithful_heart	Faithful Heart	Win 5 games as Faithful	winsAsFaithful >= 5
ghost	Ghost	Survive 5 games without being banished or murdered	totalSurvived >= 5 AND totalBanished + totalMurdered == 0 across those games — simplify: totalSurvived >= 5
scapegoat	Scapegoat	Be banished 3 times	totalBanished >= 3
veteran	Veteran	Play 10 games	gamesPlayed >= 10

Compute unlock state client-side from the `PlayerStats` object — no extra server call needed.

2. Hall of Fame screen

Add a "Hall of Fame" button to the Lobby (below the join/create controls). Tapping it sends `C2S_GET_LEADERBOARD` with metric `'winRate'`, then renders a full-screen leaderboard.

The Hall of Fame has three tab-selectable leaderboards:

- **Win Rate** — ranked by `winRate`, showing win% and games played
- **Games Played** — ranked by `gamesPlayed`, showing total wins
- **Traitor Mastery** — ranked by `traitorWinRate`, showing traitor wins / traitor games

Each leaderboard row shows:

- Rank number (1st gets a 🏆, 2nd 🥈, 3rd 🥉)
- Player name
- Primary metric value (formatted)
- Secondary info (e.g. "12 games" for win rate board)
- A subtle highlight if the row is the current player

Switching tabs sends a new `C2S_GET_LEADERBOARD` request with the appropriate metric.

At the bottom of the Hall of Fame, show the Global Stats bar: "X games played by Y players — Faithful win Z% of the time" Fetched via a new event `C2S_GET_GLOBAL_STATS` / `S2C_GLOBAL_STATS`.

Add these events to `src/game/types.ts`:

```
{ type: 'C2S_GET_GLOBAL_STATS', payload: Record<string, never> }  
{ type: 'S2C_GLOBAL_STATS', payload: GlobalStats }  
// GlobalStats: { totalGamesPlayed, totalPlayersEver, faithfulWinRate, traitorWin
```

Handle in the WS router — respond only to requesting socket.

3. Styling

Use CSS Modules. Design direction: dark, premium, slightly gothic — consistent with the game's aesthetic.

- Background: `#0d0d0d`
- Card background: `#1a1a1a`, border `#2a2a2a`
- Accent gold: `#c9a84c`

- Traitor red: #8b1a1a
- Faithful blue: #1a4a8b
- Text primary: #f0e6d3 , text secondary: #888
- Drawer slides up with `transform: translateY` transition, 300ms ease-out
- Leaderboard rows have a hover state with subtle background lift

Mobile-first layout. Drawer takes 85% of screen height. Hall of Fame is full-screen with a back button.

Constraints

- Profile drawer must not require a page navigation — it overlays the Lobby
- All stats are read-only from the client — no editing player names or resetting stats from this screen
- If no stats exist for the current player (new player), show a “No games yet” empty state
- Do not use any component library — raw HTML elements + CSS Modules only
- TypeScript strict mode must be satisfied

Definition of done

- Profile drawer opens from Lobby, shows real stats from DB
- Achievements tab shows 6 achievements with correct unlock states
- Hall of Fame shows three leaderboards with accurate data
- Global stats bar shows correct aggregates
- Current player row is highlighted on leaderboard
- TypeScript compiles cleanly, no console errors

PROMPT 5: Traitor Recruitment

You are working on **Betrayal Game** (Node.js + TypeScript backend, React 19 + Vite frontend, CSS Modules). The game has persistent player identities and stats (Wave 2 Prompts 1–4 complete).

Context

The Traitor Recruitment mechanic allows Traitors to secretly approach a Faithful player during the Night phase and offer them the chance to switch sides. This is one of the most dramatic mechanics in The Traitors TV show — it creates paranoia, alliance-shifting, and a genuinely strategic decision: *do I trust this offer, or is it a test?*

A partial task spec exists in the codebase at `.local/tasks/traitor-recruitment.md` — review it before starting and reconcile any differences with the spec below, preferring this spec where they conflict.

Game design spec

When can recruitment happen?

- Only during the Night phase, after the murder vote is resolved
- Only if: at least 1 Traitor is alive, at least 2 Faithful are alive, and the current round is ≥ 3
- Maximum 1 recruitment attempt per night
- Maximum total recruitments per game: $\text{floor}(\text{startingTraitorCount} / 2)$, minimum 1. (i.e. if the game started with 2 Traitors, only 1 recruitment is ever allowed)
- Traitors vote collectively on who to recruit — the Faithful player with the most recruitment votes is approached

The offer flow:

1. Traitors see a “Recruit a Faithful” option during Night phase, after murder votes are submitted
2. Traitors vote on which Faithful player to recruit (same vote UI pattern as murder votes)
3. When all Traitors have voted (or host force-resolves), the plurality target receives the offer privately
4. The target player sees a dramatic offer screen: “The Traitors have approached you. Join them or refuse?” with a 90-second timer
5. If they accept: their role changes to `TRAITOR`, they gain access to the Traitor chat, and the other Traitors are notified privately
6. If they refuse or the timer expires: nothing happens publicly. The approached player knows who approached them (they see the Traitor names). The Traitors do not learn the answer until the next Roundtable (if the player starts acting differently)
7. In both cases, no public announcement is made — the recruitment attempt is invisible to

uninvolved Faithful players

Win condition impact: After a successful recruitment, immediately re-check the win condition. If Traitors now equal or outnumber Faithful, the game ends (Traitors win).

Server-side implementation

New types in `src/game/types.ts`:

```
type RecruitmentVote = {
  traitorId: string
  targetId: string    // must be a Faithful player
}

type RecruitmentOffer = {
  targetPlayerId: string
  traitorIds: string[]    // who approached them – revealed to target only
  offeredAt: number       // Unix ms
  expiresAt: number       // offeredAt + 90000
  status: 'PENDING' | 'ACCEPTED' | 'REFUSED' | 'EXPIRED'
}

// Add to GameState:
recruitmentVotes: RecruitmentVote[]
currentRecruitmentOffer: RecruitmentOffer | null
totalRecruitmentsThisGame: number
```

New C2S events:

```
{ type: 'C2S_CAST_RECRUITMENT_VOTE', payload: { targetId: string } }
{ type: 'C2S_RESPOND_TO_RECRUITMENT', payload: { accepted: boolean } }
{ type: 'C2S_FORCE_RESOLVE_RECRUITMENT', payload: Record<string, never> } // hos
```

New S2C events:

```
{ type: 'S2C_RECRUITMENT_VOTE_CAST', payload: { votesReceived: number; votesNeede
// Sent only to Traitors – progress update, no target names revealed until resolv

{ type: 'S2C_RECRUITMENT_OFFER_SENT', payload: { targetPlayerId: string } }
// Sent only to Traitors – confirms who was approached

{ type: 'S2C_RECRUITMENT_OFFER_RECEIVED', payload: {
  traitorIds: string[]    // the approaching Traitor IDs
```

```

        traitorNames: string[]      // the approaching Traitor names
        expiresAt: number
    }
}
// Sent only to the target Faithful player

{ type: 'S2C_RECRUITMENT_RESOLVED', payload: {
    accepted: boolean
    targetPlayerId: string      // sent only to Traitors
}
}
// Sent only to Traitors when the offer is accepted/refused/expired

{ type: 'S2C_ROLE_CHANGED', payload: {
    newRole: 'TRAITOR'
    traitorIds: string[]      // all current Traitor IDs, sent only to the newly
}
}
// Sent only to the newly recruited player if they accept

```

New functions in `src/game/manager.ts` :

```

export function canAttemptRecruitment(gameState: GameState): boolean
// Checks: round >= 3, alive traitors >= 1, alive faithful >= 2,
// totalRecruitmentsThisGame < maxRecruitments, no current offer pending

export function castRecruitmentVote(gameState: GameState, traitorId: string, targ
// Validates: traitorId is alive Traitor, targetId is alive Faithful
// Replaces any existing vote from this traitor
// Returns updated state

export function resolveRecruitmentVotes(gameState: GameState): {
    game: GameState;
    targetPlayerId: string;
    traitorIds: string[]
}
// Tallies votes, selects plurality winner (random tiebreak)
// Creates RecruitmentOffer, sets status to PENDING
// Returns updated state + target info for broadcasting

export function respondToRecruitment(gameState: GameState, playerId: string, acce
// Validates: playerId is the offer target, offer is PENDING and not expired
// If accepted: changes player role to TRAITOR, clears recruitmentVotes, incremen
// If refused: sets offer status to REFUSED
// Returns updated state

```



```
export function expireRecruitmentOffer(gameState: GameState): GameState
// Sets offer status to EXPIRED if expiresAt has passed
// Called by a server-side setTimeout
```

Timer handling in the router:

When `S2C_RECRUITMENT_OFFER_RECEIVED` is sent, start a `setTimeout` of 90 seconds on the server. When it fires, check if the offer is still `PENDING` — if so, call

`expireRecruitmentOffer` and broadcast `S2C_RECRUITMENT_RESOLVED` with `accepted: false` to Traitors only.

Night phase flow modification:

The Night phase currently ends when murder votes are resolved and the host sends `C2S_START_MORNING`. Add a step between murder resolution and morning:

If `canAttemptRecruitment(gameState)` is true, show Traitors a prompt: "Recruit a Faithful?" with Yes/No. If Yes, enter recruitment vote flow. If No (or all Traitors decline), proceed to morning. Host can force-skip via `C2S_FORCE_RESOLVE_RECRUITMENT`.

Client-side implementation

New component: `client/src/components/RecruitmentOffer.tsx`

Shown only to the player who receives an offer. Full-screen overlay (above the NightPhase component):

- Dark background with a candle/flame motif (CSS only — a flickering pseudo-element)
- "You have been approached by the Traitors"
- Traitor names listed (this is privileged information — make it feel significant)
- A 90-second countdown timer
- Two buttons: "Join the Traitors" (crimson) and "Refuse" (muted)
- Dramatic copy: "This offer expires. Your answer is yours alone."

Modifications to `NightPhase.tsx` :

- Traitors see a "Recruit a Faithful?" prompt after murder votes are submitted
- If they choose to recruit: show a player selection list (Faithful only, alive only) with a

"Vote to Recruit" button per player

- Show progress: "X/Y Traitors have voted to recruit" (count only, no names)
- After offer is sent: "The offer has been made. Waiting..."
- When `S2C_RECRUITMENT_RESOLVED` arrives: brief result shown to Traitors only ("They joined." or "They refused.") before morning begins

Styling:

The offer screen should feel like the most dramatic moment in the game. Use:

- Background: `#080410` (near-black with purple tint)
- Accent: `#6b21a8` (deep purple — distinct from Traitor red)
- Flickering candle: CSS `@keyframes` on opacity `0.8 → 1.0 → 0.85`, 1.5s infinite alternate
- The offer text fades in slowly (1.5s ease-in)
- Accept/Refuse buttons appear only after 3 seconds (prevents accidental taps)

Constraints

- Recruitment information must be strictly scoped — uninvolved Faithful must receive zero information about the attempt
- The approached player's decision must never be revealed to anyone except Traitors (via `S2C_RECRUITMENT_RESOLVED`)
- If the recruited player accepts, their new `TRAITOR` role must be immediately reflected in chat channel access
- Do not use any animation libraries — CSS only
- TypeScript strict mode must be satisfied
- All new `GameState` fields must be initialised in `createGame` in `manager.ts`

Definition of done

- Traitors can vote to recruit during Night phase when conditions are met
- Target player receives the offer screen with correct Traitor names and 90-second timer
- Accepting changes role, grants Traitor chat access, triggers win condition check

- Refusing/expiring is invisible to uninvolved Faithful
- `S2C_RECRUITMENT_RESOLVED` reaches Traitors in both accept and refuse cases
- Offer screen buttons are delayed 3 seconds on appearance
- TypeScript compiles cleanly, no console errors
- End-to-end tested manually with 5+ players